НЮАНСЫ ПРОЕКТА

Матрица проверки: нюансы из теории Windows x64 EH/unwind -> учитывает ли их EH/Unwind Analyzer

Документ сделан по книге "итогоая теория.pdf" и по фактической реализации финальной версии проекта. Его задача - дать защитную карту: какие тонкости проект реально закрывает, где покрытие частичное, а где надо честно сказать "это не входит в универсальное ядро".

Главный вывод

Ядро проекта в целом правильно соответствует теории на уровне структурного анализа PE x64 EH/unwind metadata: PE headers, RVA/file offset, Exception Directory, .pdata/RUNTIME_FUNCTION, .xdata/UNWIND_INFO, padding, flags, chained info, handler RVA, placement/reuse heuristics, dynamic Windows evidence и отчеты. При этом проект не является универсальным парсером language-specific handler data, не является полноценным RtlVirtualUnwind emulator и не делает автоматическое дизассемблирование пролога для сравнения с UNWIND_CODE. Эти границы нужно формулировать честно.

Легенда статусов

Статус	Количество	Смысл
FULL	**62**	**нюанс книги реализован в ядре или отчетном слое**
PART	**4**	**нюанс учтен частично: есть поддержка/сигнал/ручной маршрут, но нет полной автоматизации**
OUT	**2**	**сознательная граница проекта: не заявлять как реализованное**

Как читать матрицу

Поле "Нюанс" - что книга требует понимать или не перепутать. Поле "Проверка в проекте" - где это видно в коде. Поле "Вывод" - как отвечать комиссии: реализовано, частично или находится за рамками честного scope проекта.

Границы проекта

1. Проект проверяет инфраструктуру Windows x64 exception/unwind, а не ищет бытовую "ошибку программы".

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 17-25, 99-117
Проверка в проекте	main.c:95-115, main.c:270-425, heuristics.c:19-84
Вывод для защиты	Инструмент не обещает найти все баги или всю вредоносность. Он проверяет надежность маршрута PE -> .pdata -> .xdata -> handler/dynamic evidence.

2. Статический и динамический слои отвечают на разные вопросы и не должны смешиваться.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 51, 640-655, 1136, 1145, 1148, 1229
Проверка в проекте	main.c:388-425, report.c:486-602
Вывод для защиты	Статика анализирует файл. Динамика собирает runtime-сигналы. Итоговый отчет отдельно показывает correlation и runtime evidence.

3. Целевая область - PE32+ AMD64, не x86 SEH-chain и не ARM64.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 17-25, 99-117
Проверка в проекте	pe.c:67-89, main.c:95-115
Вывод для защиты	Проект честно отвергает неподдерживаемые архитектуры, потому что правила x64 .pdata/.xdata нельзя переносить на x86.

PE и координаты

4. Нельзя путать VA, RVA и file offset.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 20-25, 99-117, 946-956
Проверка в проекте	pe.c:265-300, unwind.c:56-72, unwind.c:630-633
Вывод для защиты	RVA переводится в file offset через section table. Проект не делает seek(RVA) напрямую.

5. ImageBase нужен для связи runtime VA и image-relative RVA.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 20-25, 99-117, 946-956
Проверка в проекте	pe.c:99-105, pe.c:237-246, dynamic_win.c:789-803
Вывод для защиты	Статика хранит ImageBase, динамика складывает base_main + handler_rva для runtime breakpoint.

6. Data Directory VirtualAddress для Exception Directory - это RVA, а не абсолютный адрес.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 17-25, 32, 56, 100
Проверка в проекте	pe.c:126-137, unwind.c:630-633
Вывод для защиты	Проект читает Exception Directory из Optional Header и мапит ее как RVA.

7. Искать .pdata только по имени секции нельзя: вход через Exception Directory.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 17-25, 32, 56, 100
Проверка в проекте	pe.c:126-137, unwind.c:623-647
Вывод для защиты	Проект не зависит от имени секции .pdata для основного чтения таблицы.

8. RVA может попадать в virtual tail секции, которого физически нет в файле.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 20-25, 99-117, 946-956
Проверка в проекте	pe.c:277-300
Вывод для защиты	Если delta >= raw_size, проект отказывается читать байты, а не подставляет нули как файл.

9. Секции должны иметь корректные raw/virtual диапазоны.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 17-25, 99-117
Проверка в проекте	pe.c:198-228, heuristics.c:789-824
Вывод для защиты	Считаются raw_oob, virtual_oob, raw/virtual overlaps и превращаются в findings.

10. FileAlignment/SectionAlignment/SizeOfHeaders/SizeOfImage должны быть согласованы.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 17-25, 99-117
Проверка в проекте	pe.c:92-124, pe.c:180-196, heuristics.c:817-824
Вывод для защиты	Нарушения не замалчиваются: попадают в pe_section_misaligned.

11. Права секций важны: handler должен быть кодом, unwind metadata обычно не должна быть writable/executable.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 40, 45-49, 84, 89-90, 119-120, 178
Проверка в проекте	heuristics.c:118-143, heuristics.c:690-723, heuristics.c:964-1000
Вывод для защиты	Проект проверяет placement по секции и правам, а не только по числам RVA.

12. PE-заголовки нельзя читать без проверки границ.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 17-25, 99-117
Проверка в проекте	pe.c:40-89, pe.c:139-148
Вывод для защиты	Проверяются MZ, PE signature, COFF, Optional Header size, section headers size.

.pdata / RUNTIME_FUNCTION

13. Exception Directory может корректно отсутствовать только как RVA=0 и Size=0.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 17-25, 32, 56, 100
Проверка в проекте	unwind.c:623-628, heuristics.c:833-841
Вывод для защиты	Несогласованная пара RVA/Size считается malformed.

14. Размер Exception Directory должен быть не меньше одной записи и кратен 12 байтам.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 20, 26-33, 86, 94, 99
Проверка в проекте	unwind.c:639-647
Вывод для защиты	Проект проверяет размер и кратность sizeof(RUNTIME_FUNCTION)=12.

15. BeginAddress, EndAddress, UnwindInfoAddress являются RVA.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 20, 26-33, 86, 94, 99
Проверка в проекте	unwind.c:654-660, heuristics.c:652-676
Вывод для защиты	Записи читаются как 32-bit RVA, затем проверяются относительно SizeOfImage.

16. BeginAddress должен быть меньше EndAddress.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 20, 26-33, 86, 94, 99
Проверка в проекте	heuristics.c:652-656, heuristics.c:850-856
Вывод для защиты	Нарушение становится finding invalid_ranges.

17. Диапазон функции должен попадать внутрь SizeOfImage.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 20, 26-33, 86, 94, 99
Проверка в проекте	heuristics.c:657-662, heuristics.c:857-863
Вывод для защиты	Нарушение становится finding rf_oob.

18. UnwindInfoAddress должен указывать внутрь image и не быть нулевым.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 20, 26-33, 86, 94, 99
Проверка в проекте	heuristics.c:663-667, heuristics.c:864-870
Вывод для защиты	Нарушение становится finding unwind_rva_oob.

19. Таблица должна быть отсортирована по BeginAddress.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 20, 26-33, 86, 94, 99
Проверка в проекте	heuristics.c:669-676, heuristics.c:871-877
Вывод для защиты	Проект считает inversions и выдает rf_unsorted.

20. Диапазоны функций не должны пересекаться.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 20, 26-33, 86, 94, 99
Проверка в проекте	heuristics.c:740-773, heuristics.c:878-884
Вывод для защиты	Проект сортирует копию диапазонов и ищет overlaps независимо от исходного порядка.

21. Повтор BeginAddress подозрителен.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 20, 26-33, 86, 94, 99
Проверка в проекте	heuristics.c:757-763, heuristics.c:885-891
Вывод для защиты	Проект выдает duplicate_begin.

22. Отсутствие .pdata не всегда ошибка: leaf-only/resource-only случаи возможны.

Статус	**PART - частично учтено**
Где в книге	книга: стр. PDF 20, 26-33, 86, 94, 99
Проверка в проекте	heuristics.c:833-841
Вывод для защиты	Проект отличает пустую .pdata при executable-секциях от resource-only случая, но не дизассемблирует весь код, чтобы доказать leaf/non-leaf по каждой функции.

.xdata / UNWIND_INFO

23. UnwindInfoAddress должен мапиться в file offset перед чтением.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 36, 208-234, 289-299
Проверка в проекте	unwind.c:56-72, unwind.c:187-195
Вывод для защиты	Если RVA не мапится или заголовок обрезан, это parse failure.

24. UNWIND_INFO должен иметь поддерживаемую версию.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 36, 208-234, 289-299
Проверка в проекте	unwind.c:197-214
Вывод для защиты	Поддерживаются версии 1 и 2; неизвестная версия дает UA_VERSION_UNKNOWN.

25. Флаги EHANDLER/UHANDLER/CHAININFO меняют смысл хвоста структуры.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 40, 45-49, 84, 89-90, 119-120, 178
Проверка в проекте	unwind.c:197-220, unwind.c:519-552
Вывод для защиты	Проект читает tail как handler RVA или chained RUNTIME_FUNCTION в зависимости от flags.

26. CHAININFO не должен совмещаться с EHANDLER/UHANDLER.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 40, 89, 198, 209, 213, 221-222, 346, 351
Проверка в проекте	unwind.c:215-220
Вывод для защиты	Недопустимое сочетание помечается UA_BAD_OPERAND.

27. CountOfCodes - это число 2-byte slots, а не число высокоуровневых операций.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 36, 208-234, 289-299
Проверка в проекте	unwind.c:24-54, unwind.c:251-275
Вывод для защиты	Проект знает длину каждой UWOP-операции и не идет по массиву фиксированно по 1 операции.

28. После нечетного CountOfCodes есть padding slot перед handler/chained tail.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 36, 208, 213, 221, 289-299, 590, 607
Проверка в проекте	unwind.c:231-235, unwind.c:519-543
Вывод для защиты	Используется slots_even=(count+1)&~1; tail читается от after_codes после padding.

29. Нельзя позволять opcode занимать слоты за пределами CountOfCodes.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 36, 208-234, 289-299
Проверка в проекте	unwind.c:265-275
Вывод для защиты	Если i+slots > count, проект ставит UA_TRUNCATED и прекращает парсинг записи.

30. CodeOffset должен относиться к прологу и идти в корректном порядке.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 36, 208-234, 289-299
Проверка в проекте	unwind.c:276-286
Вывод для защиты	Проект ставит UA_CODEOFFSET_OUTSIDE и UA_CODEOFFSET_ORDER.

31. UWOP_ALLOC_LARGE имеет два режима и разные размеры operands.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 36, 208-234, 289-299
Проверка в проекте	unwind.c:31-33, unwind.c:357-381
Вывод для защиты	Mode 0 читает один дополнительный slot и масштабирует на 8; mode 1 читает 32-bit size.

32. UWOP_ALLOC_SMALL дает размер opinfo*8+8.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 36, 208-234, 289-299
Проверка в проекте	unwind.c:383-385
Вывод для защиты	Проект добавляет фиксированное выделение стека по правилу кодировки.

33. SAVE_NONVOL/SAVE_XMM и FAR-варианты занимают разное число slots.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 36, 208-234, 289-299
Проверка в проекте	unwind.c:35-49, unwind.c:394-502
Вывод для защиты	Проект читает scaled/lo/hi operands и проверяет подозрительные offsets.

34. SET_FPREG имеет смысл только при заданном frame register.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 36, 208-234, 289-299
Проверка в проекте	unwind.c:221-228, unwind.c:386-392, unwind.c:515-517
Вывод для защиты	Нарушение помечается UA_BAD_OPERAND; strict ABI добавляет reserved-opinfo warning.

35. RSP нельзя считать обычным сохраняемым GPR.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 36, 208-234, 289-299
Проверка в проекте	unwind.c:86-100, unwind.c:348-356
Вывод для защиты	is_unwindable_gpr исключает register 4 (RSP).

36. PUSH_MACHFRAME имеет только допустимые opinfo 0 или 1.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 36, 208-234, 289-299
Проверка в проекте	unwind.c:503-506
Вывод для защиты	Иначе выставляется UA_BAD_OPERAND; размер стека 40 или 48 байт.

37. Сумма fixed stack не должна переполняться.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 36, 208-234, 289-299
Проверка в проекте	unwind.c:103-111
Вывод для защиты	При overflow fixed_stack saturates до UINT32_MAX и ставится UA_OPERAND_SANITY.

38. Version 2 epilog/spare descriptors нельзя читать как обычные v1 XMM-сохранения.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 36, 208-234, 289-299
Проверка в проекте	unwind.c:37-47, unwind.c:306-337, unwind.c:429-467
Вывод для защиты	Проект различает v2 op 6/7 и v1 undocumented XMM variants.

39. ABI-предупреждения не равны автоматически сильному вмешательству.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 36, 208-234, 289-299
Проверка в проекте	unwind.c:287-295, heuristics.c:14-16, heuristics.c:727-735
Вывод для защиты	ABI bits отделены через UA_ABI_MASK и считаются отдельно от hard unwind anomalies.

40. Проект не делает полную эмуляцию RtlVirtualUnwind и не вычисляет caller context для конкретного live ControlPc.

Статус	**OUT - не входит в универсальное ядро**
Где в книге	книга: стр. PDF 14, 34, 41-50, 3973-4042
Проверка в проекте	unwind.c:554-607, report.c:625-675
Вывод для защиты	Проект строит structural summary и findings. Полный RtlVirtualUnwind emulator - отдельное расширение, не обязательное для заявленного structural analyzer.

41. Проект не дизассемблирует реальный пролог и не сравнивает машинные инструкции с UNWIND_CODE автоматически.

Статус	**PART - частично учтено**
Где в книге	книга: стр. PDF 36, 208-234, 289-299
Проверка в проекте	report.c:625-640, ghidra/import_unwind_csv.py
Вывод для защиты	Проект декодирует UNWIND_CODE и готовит Ghidra/manual validation, но автоматического disassembler-сравнения пока нет.

Handler / chain

42. Handler RVA читается после корректно выровненного массива UNWIND_CODE.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 36, 208, 213, 221, 289-299, 590, 607
Проверка в проекте	unwind.c:231-235, unwind.c:519-552
Вывод для защиты	Это защищает от сдвига на 2 байта при нечетном CountOfCodes.

43. Handler RVA должен находиться внутри SizeOfImage.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 40, 45-49, 84, 89-90, 119-120, 178
Проверка в проекте	unwind.c:540-550
Вывод для защиты	Нарушение дает UA_HANDLER_OOB.

44. Handler target должен указывать на исполняемый код, writable/nonexec placement подозрителен.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 40, 45-49, 84, 89-90, 119-120, 178
Проверка в проекте	heuristics.c:714-723, heuristics.c:988-1000
Вывод для защиты	Проект проверяет секцию handler-а и формирует handler_nonexec_section / handler_writable_section.

45. Один handler, обслуживающий слишком много функций, может быть normal runtime helper или dispatcher-style признаком.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 40, 45-49, 84, 89-90, 119-120, 178
Проверка в проекте	heuristics.c:782-785, heuristics.c:920-943
Вывод для защиты	Проект считает handler_reuse_ratio и добавляет handler_dominance только с контекстным ослаблением.

46. Language-specific handler data после handler RVA нельзя универсально парсить без знания handler-а.

Статус	**OUT - не входит в универсальное ядро**
Где в книге	книга: стр. PDF 40, 45-49, 84, 89-90, 119-120, 178
Проверка в проекте	unwind.c:540-552, report.c:667-668
Вывод для защиты	Проект учитывает границу: читает handler RVA, placement, dominance и dynamic hits, но не парсит scope table или C++ EH data универсально.

47. CHAININFO tail - это RUNTIME_FUNCTION-like запись из трех RVA.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 40, 89, 198, 209, 213, 221-222, 346, 351
Проверка в проекте	unwind.c:130-140, unwind.c:522-537
Вывод для защиты	Проект читает Begin/End/UnwindRVA chained entry и валидирует поля.

48. Chained unwind может зациклиться или быть слишком глубоким.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 40, 89, 198, 209, 213, 221-222, 346, 351
Проверка в проекте	main.c:107-109, main.c:212-219, unwind.c:559-604
Вывод для защиты	Есть --max-chain-depth 1..16, loop detection и UA_CHAIN_DEPTH_EXCEEDED.

49. Chained unwind влияет на summary стека/регистров.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 40, 89, 198, 209, 213, 221-222, 346, 351
Проверка в проекте	unwind.c:586-597
Вывод для защиты	Проект объединяет fixed_stack_bytes, pushed count, saved masks и handler из цепочки.

50. Точная логика вызова handler зависит от положения ControlPc в prolog/body/epilog.

Статус	**PART - частично учтено**
Где в книге	книга: стр. PDF 40, 45-49, 84, 89-90, 119-120, 178
Проверка в проекте	unwind.c:197-552, report.c:625-675
Вывод для защиты	Проект анализирует metadata структуры, но не моделирует конкретный exception dispatch для каждого ControlPc внутри функции.

Эвристики

51. Нельзя кричать "malware" по одной странности.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 51, 640-655, 1136, 1145, 1148, 1229
Проверка в проекте	heuristics.c:19-84, heuristics.c:521-602
Вывод для защиты	Score/confidence/evidence_level разделены; есть weakening rules.

52. Признаки должны группироваться по семействам доказательств.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 912, 936-938, 1027, 1060-1104
Проверка в проекте	heuristics.c:367-395, report.c:470-483
Вывод для защиты	Используются table_integrity, unwind_integrity, metadata_placement, unwind_reuse, handler_dominance.

53. Комбинации независимых признаков сильнее одиночных сигналов.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 912, 936-938, 1027, 1060-1104
Проверка в проекте	heuristics.c:1003-1023
Вывод для защиты	Проект добавляет combo findings: placement+integrity, reuse+dominance, writable metadata+handler.

54. Малый sample size и одиночный сигнал должны снижать уверенность.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 912, 936-938, 1027, 1060-1104
Проверка в проекте	heuristics.c:521-602
Вывод для защиты	Есть mitigations single_signal_only, small_sample_size, placement_only_signal, single_family_only.

55. Результат должен объяснять observed/expected/impact, а не только число score.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 912, 936-938, 1027, 1060-1104
Проверка в проекте	report.c:60-145, report.c:625-675
Вывод для защиты	HTML/JSON содержат rule/expected/impact и manual validation steps.

Динамика

56. Dynamic layer не заменяет static parser, а подтверждает runtime-поведение.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 51, 640-655, 1136, 1145, 1148, 1229
Проверка в проекте	dynamic.h:3-14, report.c:486-602
Вывод для защиты	DynamicReport дополняет static report и оценивается отдельно.

57. Динамика Windows-only, потому что использует Debug API.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 51, 640-655, 1136, 1145, 1148, 1229
Проверка в проекте	dynamic.h:95-118, dynamic_win.c:700-713
Вывод для защиты	Это честная ОС-граница. На macOS/Linux статическая часть работает, динамическая Windows-валидация нет.

58. Нужно различать first chance и second chance.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 51, 640-655, 1136, 1145, 1148, 1229
Проверка в проекте	dynamic_win.c:832-849, dynamic.h:63-65
Вывод для защиты	Проект считает first_chance_seen и second_chance_seen.

59. VEH/VCH/UEF - отдельные runtime-механизмы, не то же самое, что .pdata handler.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 51, 640-655, 1136, 1145, 1148, 1229
Проверка в проекте	dynamic.h:70-75, dynamic_win.c:902-949, dynamic_win.c:997-1028
Вывод для защиты	Проект ловит регистрации и hits AddVectoredExceptionHandler, AddVectoredContinueHandler, SetUnhandledExceptionFilter.

60. Handler-и из UNWIND_INFO можно проверить через runtime breakpoint.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 40, 45-49, 84, 89-90, 119-120, 178
Проверка в проекте	dynamic_win.c:726-746, dynamic_win.c:798-808, dynamic_win.c:984-995
Вывод для защиты	Проект собирает уникальные handler_rva и ставит breakpoints на base_main+handler_rva.

61. JIT/runtime может регистрировать function tables через RtlAddFunctionTable или callback.

Статус	**PART - частично учтено**
Где в книге	книга: стр. PDF 31, 104, 159, 172, 198, 413, 424, 517, 711
Проверка в проекте	dynamic.h:77-78, dynamic_win.c:950-983, report.c:657-660
Вывод для защиты	Проект ловит вызовы и аргументы, но не делает полный dump/parse содержимого runtime table.

62. Отсутствие динамического события не доказывает отсутствие механизма.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 51, 640-655, 1136, 1145, 1148, 1229
Проверка в проекте	dynamic.h:41-89, report.c:522-534, report.c:671-672
Вывод для защиты	Отчет учитывает timeout, instrumentation_complete, target_exited и рекомендует повтор на representative exception path.

63. После handler hit полезен короткий trace RIP.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 51, 640-655, 1136, 1145, 1148, 1229
Проверка в проекте	main.c:110-112, dynamic.h:84-86, dynamic_win.c:992-1028
Вывод для защиты	Проект хранит trace_steps и samples после handler/VEH/VCH/UEF hit.

Отчеты и интеграция

64. JSON должен быть главным машинным контрактом для backend/database.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 912, 936-938, 1027, 1060-1104
Проверка в проекте	main.c:450-460, report.c:800-1030
Вывод для защиты	В JSON идут metadata, functions, heuristics, dynamic assessment.

65. Отчет должен сохранять provenance: версия, schema, command line, SHA-256 input.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 912, 936-938, 1027, 1060-1104
Проверка в проекте	main.c:450-460, report.c:800-1030
Вывод для защиты	Проект готовит ReportMetadata перед JSON/HTML.

66. CSV нужен для табличной проверки, Ghidra CSV - для разметки дизассемблера.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 912, 936-938, 1027, 1060-1104
Проверка в проекте	main.c:427-447, report.c:300-380, ghidra/import_unwind_csv.py
Вывод для защиты	Есть обычный CSV и специальный begin_rva/end_rva/name/comment для Ghidra.

67. Ghidra import не должен ломать существующую разметку.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 912, 936-938, 1027, 1060-1104
Проверка в проекте	ghidra/import_unwind_csv.py
Вывод для защиты	Скрипт сохраняет пользовательские имена/комментарии, пропускает overlaps и добавляет bookmarks на warnings.

68. HTML нужен для человека, но source of truth остается structured analysis.

Статус	**FULL - учтено**
Где в книге	книга: стр. PDF 912, 936-938, 1027, 1060-1104
Проверка в проекте	main.c:457-460, report.c
Вывод для защиты	Проект пишет HTML как читаемое представление тех же данных, что уходят в JSON.

Критичные честные формулировки для защиты

Про language-specific handler data

Проект видит факт наличия handler-a и читает handler RVA, но не универсально разбирает данные после handler RVA. Это правильно, потому что scope tables и C++ EH structures зависят от конкретного language/runtime handler-a.

Про padding

Проект учитывает padding после нечетного CountOfCodes через округление количества slots до четного. Поэтому handler RVA и chained info читаются с правильного смещения.

Про JIT

Static layer анализирует PE-файл на диске. Runtime/JIT function tables появляются во время исполнения. Dynamic layer фиксирует RtlAddFunctionTable и RtlInstallFunctionTableCallback, но полный dump/parse runtime table - возможное расширение.

Про точность

Это не ML-модель и не бинарный классификатор malware/benign. Это объяснимая система признаков: score показывает силу найденных нарушений, confidence - надежность интерпретации, а manual review требуется при неоднозначных случаях.

Про завершенность ядра

Для заявленной области "структурный анализ и выявление аномалий PE x64 EH/unwind metadata" ядро закрывает основные инварианты. Не надо заявлять, что оно полностью понимает все языковые runtimes или полностью эмулирует Windows unwinder.

Что можно допилить в будущем без признания текущего ядра неправильным

- Добавить optional parsers для известных language-specific formats: __C_specific_handler scope table, __CxxFrameHandler3/4 structures, если заранее определен handler type.

- Добавить disassembler-backed comparison: сравнивать UNWIND_CODE с реальными инструкциями пролога в .text.

- В dynamic layer читать память runtime RtlAddFunctionTable table pointer и сохранять snapshot registered RUNTIME_FUNCTION entries.

- Добавить partial RtlVirtualUnwind emulator для учебной валидации selected ControlPc, но отделить его от основного structural verdict.

- Добавить больше corpus-тестов на language/runtime-specific binaries: MSVC C SEH, MSVC C++ EH, Rust, Go, JIT-like samples.

Итог

Если комиссия спросит "учитывает ли проект все тонкости из теории", правильный ответ такой: проект учитывает основные универсальные тонкости PE x64 EH/unwind формата и честно отделяет их от runtime/language-specific областей. Полностью закрыты координаты PE, чтение Exception Directory, структура RUNTIME_FUNCTION, UNWIND_INFO, padding, flags, chain, handler RVA, placement/reuse heuristics, dynamic evidence и reports. Частично закрыты JIT runtime tables, leaf/non-leaf inference, prolog-vs-UNWIND_CODE comparison и concrete ControlPc dispatch modeling. Не входит в универсальное ядро полный разбор language-specific handler data без знания конкретного handler-a.